

# Encapsulamento e Polimorfismo em Python

Aprofundamento — de onde você parou até aplicação prática

Material de estudo — Edy / Cleitin

Agosto de 2026

# Sumário

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>Antes de começar</b>                                                          | <b>3</b>  |
| <b>PARTE 1 — ENCAPSULAMENTO</b>                                                  | <b>4</b>  |
| <b>Capítulo 1 — Por que encapsular (motivação real, não decoreba)</b>            | <b>5</b>  |
| O problema que encapsulamento resolve . . . . .                                  | 5         |
| Analogia . . . . .                                                               | 5         |
| Os 4 níveis de “visibilidade” que Python realmente oferece . . . . .             | 5         |
| Erros comuns . . . . .                                                           | 6         |
| Exercícios . . . . .                                                             | 7         |
| <b>Capítulo 2 — @property: o jeito Pythonico de fazer getter/setter</b>          | <b>8</b>  |
| Conceito . . . . .                                                               | 8         |
| @property sem setter — atributo somente leitura . . . . .                        | 9         |
| @x.deleter (menos comum, mas existe) . . . . .                                   | 9         |
| Erros comuns . . . . .                                                           | 9         |
| Exercícios . . . . .                                                             | 10        |
| <b>Capítulo 3 — Encapsulamento na prática: invariantes e validação robusta</b>   | <b>11</b> |
| Conceito . . . . .                                                               | 11        |
| Encapsulamento não é só “esconder”, é também “simplificar a interface” . . . . . | 12        |
| Erros comuns . . . . .                                                           | 12        |
| Exercícios . . . . .                                                             | 12        |
| <b>Miniprojeto — Parte 1: Sistema de Check-in de Academia</b>                    | <b>13</b> |
| Especificação . . . . .                                                          | 13        |
| <b>PARTE 2 — POLIMORFISMO</b>                                                    | <b>14</b> |
| <b>Capítulo 4 — Duck typing: polimorfismo sem hierarquia</b>                     | <b>15</b> |
| Conceito . . . . .                                                               | 15        |
| Duck typing vs polimorfismo por herança — quando usar cada um . . . . .          | 15        |
| hasattr e EAFP vs LBYL . . . . .                                                 | 16        |
| Erros comuns . . . . .                                                           | 16        |
| Exercícios . . . . .                                                             | 17        |
| <b>Capítulo 5 — Sobrescrita de métodos (override) e polimorfismo com herança</b> | <b>18</b> |
| Conceito . . . . .                                                               | 18        |
| Cuidado: mudar a assinatura na sobrescrita quebra o polimorfismo . . . . .       | 18        |
| isinstance() e issubclass() — quando checar tipo é legítimo . . . . .            | 19        |
| Erros comuns . . . . .                                                           | 19        |
| Exercícios . . . . .                                                             | 19        |

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>Capítulo 6 — Sobrecarga de operadores (operator overloading)</b>                | <b>20</b> |
| Conceito . . . . .                                                                 | 20        |
| NotImplemented vs NotImplementedError — não confunda . . . . .                     | 21        |
| Tabela de dunder methods de operadores mais usados . . . . .                       | 21        |
| Erros comuns . . . . .                                                             | 22        |
| Exercícios . . . . .                                                               | 22        |
| <b>Capítulo 7 — Polimorfismo com ABC: contrato formal + comportamento variável</b> | <b>23</b> |
| Conceito . . . . .                                                                 | 23        |
| Erros comuns . . . . .                                                             | 24        |
| Exercícios . . . . .                                                               | 24        |
| <b>Projeto integrador final — Encapsulamento + Polimorfismo juntos</b>             | <b>25</b> |
| Especificação: Sistema de Ranking de Academia . . . . .                            | 25        |
| Checklist de autoavaliação . . . . .                                               | 25        |
| <b>Apêndice — Gabarito comentado (só depois de tentar)</b>                         | <b>27</b> |
| Capítulo 1 . . . . .                                                               | 27        |
| Capítulo 2 . . . . .                                                               | 27        |
| Capítulo 4 . . . . .                                                               | 28        |
| Capítulo 6 . . . . .                                                               | 28        |

# Antes de começar

Esse documento assume que você já passou pelos capítulos 1 a 4 do curso principal (classes, objetos, `__init__`, métodos de classe/estáticos) e leu o resumo de encapsulamento no capítulo 5. Aqui a gente vai fundo mesmo — sem atalho, sem pular a parte chata que é onde o entendimento real se consolida.

Estrutura: primeiro fechamos Encapsulamento até o fim (porque é onde você parou), com um miniprojeto de fixação. Depois entramos em Polimorfismo do zero até sobrecarga de operadores, com outro miniprojeto. No final, um projeto único que **exige os dois pilares juntos** — porque na prática eles quase sempre aparecem combinados.

# **PARTE 1 — ENCAPSULAMENTO**

# Capítulo 1 — Por que encapsular (motivação real, não decoreba)

## O problema que encapsulamento resolve

Sem encapsulamento, qualquer parte do seu programa pode colocar um objeto num estado inconsistente:

```
class ContaTreino:
    def __init__(self, saldo_creditos):
        self.saldo_creditos = saldo_creditos

conta = ContaTreino(10)
conta.saldo_creditos = -500 # nada impede isso. Nada.
```

Isso parece bobo num exemplo de 5 linhas, mas em um sistema real — com múltiplos arquivos, múltiplos desenvolvedores, código que roda daqui a 8 meses e que você não lembra mais dos detalhes — esse tipo de acesso irrestrito é a origem de boa parte dos bugs difíceis de rastrear: o objeto ficou num estado impossível e você não sabe **onde** isso aconteceu, porque podia ter sido em qualquer lugar do código.

Encapsulamento é sobre **reduzir a superfície de onde um bug desse tipo pode se originar**, forçando toda alteração de estado a passar por um ponto controlado (um método ou uma property) que pode validar.

## Analogia

Pensa em carga máxima de um exercício. Você não deixa qualquer pessoa destravar o rack e trocar a carga do jeito que quiser sem passar pelo protocolo de segurança (checar se tem spotter, se o cinto tá ajustado). O “setter” é o protocolo: ele intercepta a tentativa de mudança e só deixa passar se as condições de segurança forem satisfeitas.

## Os 4 níveis de “visibilidade” que Python realmente oferece

Python **não tem** encapsulamento forçado pelo interpretador como `private` em Java. É convenção + um mecanismo técnico leve (name mangling). Entender isso de verdade é o primeiro passo pra parar de tratar `__atributo` como “cadeado inquebrável” — não é.

| Sintaxe  | Nome                      | O que realmente acontece                                                                                                                                                                  |
|----------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nome     | Público                   | Acesso livre, sem restrição nenhuma.                                                                                                                                                      |
| _nome    | “Protegido” (convenção)   | Nada tecnicamente diferente — é um sinal pra outros programadores: “isso é detalhe interno, não mexe de fora”. O interpretador não bloqueia nada.                                         |
| __nome   | “Privado” (name mangling) | Python <b>renomeia</b> o atributo internamente para <code>_NomeDaClasse__nome</code> , dificultando (não impedindo) acesso acidental de fora e evitando colisão de nomes em herança.      |
| __nome__ | Dunder / mágico           | <b>Não é privado.</b> É a convenção reservada para os métodos especiais do próprio Python (capítulo 9 do curso principal). Nunca invente seus próprios atributos <code>__assim__</code> . |

```
class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome           # público
        self._peso = peso          # protegido por convenção
        self.__id_secreto = 999    # name mangling

joao = Atleta("João", 82)
print(joao.nome)                 # OK, normal
print(joao._peso)                 # funciona! Python não bloqueia. Mas é um "não faça isso"
# print(joao.__id_secreto)        # AttributeError!
print(joao._Atleta__id_secreto)   # 999 – funciona, porque é isso que o mangling faz
```

**Ponto crítico:** name mangling não é segurança, é **prevenção de colisão de nomes em herança** — o caso de uso real é evitar que uma subclasse acidentalmente sobrescreva um atributo “privado” da superclasse com o mesmo nome. Não trate `__x` como criptografia.

## Erros comuns

- Achar que `__atributo` é inacessível de fora — é só mais difícil e feio de acessar, não impossível.
- Usar `__` (duplo underscore) em tudo por “segurança” quando `_` (simples) já comunica a intenção suficientemente bem para 95% dos casos. Duplo underscore tem um propósito técnico específico (evitar colisão em herança), não é “mais privado ainda”.

## Exercícios

**1.1** — Crie uma classe `Cofre` com `__init__(self, senha)` guardando a senha como `__senha` (name mangling). Tente acessar `cofre.__senha` de fora e leia o erro. Depois acesse via `cofre._Cofre__senha` e confirme que funciona. Escreva, em uma frase, por que isso prova que não é “privado de verdade”.

**1.2** — Reescreva a classe `ContaTreino` do início do capítulo para usar `_saldo_creditos` (convenção de protegido) e adicione os métodos `adicionar_creditos(self, qtd)` e `usar_credito(self)`, ambos validando que o saldo nunca fica negativo (lance `ValueError` se a operação levaria a saldo negativo).

# Capítulo 2 — @property: o jeito Pythonico de fazer getter/setter

## Conceito

Em linguagens como Java, o padrão é: atributo sempre privado, e você escreve `getPeso()`/`setPeso()` manualmente pra tudo, desde o dia 1 — mesmo quando não tem validação nenhuma ainda.

Python inverte essa lógica: **comece com atributo público simples**. Se, depois, você precisar de validação ou lógica no acesso, converta pra `@property` — e o mais importante: **o código que já usa `objeto.peso = 90` continua funcionando exatamente igual**, porque a sintaxe de acesso a atributo e a sintaxe de chamada de `property` são idênticas por fora.

```
class Atleta:
    def __init__(self, nome, peso):
        self.nome = nome
        self.peso = peso  # aqui já está passando pelo setter, veja abaixo

    @property
    def peso(self):
        """Getter — chamado quando você faz atleta.peso"""
        return self._peso

    @peso.setter
    def peso(self, valor):
        """Setter — chamado quando você faz atleta.peso = valor"""
        if not isinstance(valor, (int, float)):
            raise TypeError("Peso precisa ser numérico")
        if valor <= 0:
            raise ValueError("Peso precisa ser positivo")
        self._peso = valor

joao = Atleta("João", 82)
print(joao.peso)  # 82 — chama o getter, mas parece acesso normal a atributo
joao.peso = 90    # chama o setter, valida antes de aceitar
# joao.peso = -10 # ValueError!
```

**Detalhe importante que costuma confundir:** dentro do `__init__`, `self.peso = peso` já está chamando o setter — porque `peso` virou uma `property` da classe inteira, não importa de onde você atribui a ela. É assim que a validação se aplica também na criação do objeto, não só em atribuições posteriores.

## @property sem setter — atributo somente leitura

```
class Atleta:
    def __init__(self, nome, peso, altura):
        self.nome = nome
        self.peso = peso
        self.altura = altura

    @property
    def imc(self):
        """Só getter — não existe 'imc.setter'. Não dá pra fazer atleta.imc = 25."""
        return round(self.peso / (self.altura ** 2), 1)

joao = Atleta("João", 82, 1.78)
print(joao.imc)      # 25.9 — calculado sob demanda, sempre atualizado
# joao.imc = 25      # AttributeError: can't set attribute
```

Isso é poderoso: imc nunca fica “desatualizado” porque não é armazenado, é **calculado toda vez que é acessado**, a partir do estado atual (peso, altura).

## @x.deleter (menos comum, mas existe)

```
class Atleta:
    @property
    def peso(self):
        return self._peso

    @peso.setter
    def peso(self, valor):
        self._peso = valor

    @peso.deleter
    def peso(self):
        print("Removendo registro de peso")
        del self._peso
```

## Erros comuns

- Nomear o atributo interno igual ao da property (`self.peso = peso` dentro do getter/setter de peso) — isso causa **recursão infinita**, porque `self.peso = valor` dentro do próprio setter de peso chama o setter de novo. O padrão correto é sempre guardar internamente com underscore (`self._peso`), diferente do nome da property (peso).

```
# ERRADO — recursão infinita, RecursionError
class AtletaBuggy:
    @property
    def peso(self):
        return self.peso # chama a si mesmo pra sempre

    @peso.setter
    def peso(self, valor):
```

```
self.peso = valor # idem — nunca termina
```

- Esquecer que property só funciona assim em classes que herdam de object — o que, em Python 3, é **toda classe**, então na prática esse erro só existia em Python 2 legado. Bom saber que existiu, não precisa se preocupar mais.

## Exercícios

**2.1** — Reescreva a classe Atleta usando @property/@peso.setter para validar que peso > 0 e altura > 0. Tente criar um atleta com peso negativo e confirme que o erro acontece **na criação**, não silenciosamente.

**2.2** — Adicione uma property somente-leitura classificacao\_imc que retorna a string (“abaixo do peso”/“peso normal”/“sobrepeso”/“obesidade”) calculada a partir de imc — reaproveite a lógica do @staticmethod do capítulo 4 do curso principal, mas agora chamada internamente pela property.

**2.3 (a pegadinha da recursão)** — Escreva de propósito uma property com o bug de recursão infinita descrito acima, rode, deixe estourar RecursionError, leia o traceback até entender exatamente onde ele fica “girando”, e então corrija.

**2.4 (desafio)** — Crie uma classe PlanoDeTreino com uma property carga\_total (somente leitura, calculada a partir da lista de exercícios) e uma property nivel\_intensidade (somente leitura) que retorna “leve”, “moderado” ou “intenso” com base em faixas de carga\_total. Adicione um exercício à lista depois de criar o plano e confirme que nivel\_intensidade reflete a mudança automaticamente, sem precisar recalcular nada manualmente.

# Capítulo 3 — Encapsulamento na prática: invariantes e validação robusta

## Conceito

“Invariante” é uma condição que **deve ser sempre verdadeira** para o objeto estar num estado válido — por exemplo, “peso é sempre positivo” ou “nível de treino é sempre um entre um conjunto fixo de valores válidos”. Encapsulamento bem feito é essencialmente: identificar as invariantes do seu domínio e garantir, via setters/properties, que **é impossível** violá-las através da interface pública do objeto.

```
class Atleta:
    NIVEIS_VALIDOS = {"iniciante", "intermediário", "avançado"}

    def __init__(self, nome, peso, nivel="iniciante"):
        self.nome = nome
        self.peso = peso
        self.nivel = nivel

    @property
    def nivel(self):
        return self._nivel

    @nivel.setter
    def nivel(self, valor):
        if valor not in self.NIVEIS_VALIDOS:
            raise ValueError(
                f"Nível inválido: {valor!r}. Válidos: {self.NIVEIS_VALIDOS}"
            )
        self._nivel = valor

    @property
    def peso(self):
        return self._peso

    @peso.setter
    def peso(self, valor):
        if not isinstance(valor, (int, float)) or valor <= 0:
            raise ValueError("Peso precisa ser um número positivo")
        self._peso = valor
```

Note que **cada invariante tem seu próprio setter cuidando dela** — isso é encapsulamento fazendo seu trabalho: o objeto se protege sozinho, e qualquer código em qualquer lugar do sistema que tentar violar a regra vai tomar erro imediatamente, na origem, em vez de o bug aparecer silenciosamente três camadas depois.

## Encapsulamento não é só “esconder”, é também “simplificar a interface”

Outro papel importante do encapsulamento: esconder **complexidade de implementação**, não só proteger contra valores inválidos.

```
class PlanoDeTreino:
    def __init__(self):
        self._exercicios_por_dia = {} # detalhe interno: dict de listas

    def adicionar_exercicio(self, dia_semana, exercicio):
        """Interface pública simples — quem usa não precisa saber que por dentro é um dict
        if dia_semana not in self._exercicios_por_dia:
            self._exercicios_por_dia[dia_semana] = []
            self._exercicios_por_dia[dia_semana].append(exercicio)

    def exercicios_do_dia(self, dia_semana):
        return self._exercicios_por_dia.get(dia_semana, [])
```

Amanhã você pode trocar `_exercicios_por_dia` de dict para um banco de dados, ou pra uma estrutura mais sofisticada, e **nenhum código externo que usa `adicionar_exercicio/exercicios_do_dia` precisa mudar** — essa é a vantagem prática real de encapsular: você ganha liberdade pra mudar a implementação interna sem quebrar quem depende da classe.

## Erros comuns

- Validar só no `__init__` e esquecer que o mesmo atributo pode ser alterado depois — se você não usar `@property`, `atleta.peso = -50` depois de criado passa batido mesmo que o construtor tenha validado direitinho no início.
- Expor demais: criar getters/setters públicos para **todo** atributo, inclusive detalhes que realmente não deveriam ser acessíveis de fora. Encapsular bem às vezes significa **não** criar um jeito de acessar algo de fora, e tudo bem.

## Exercícios

**3.1** — Na classe `Atleta` acima, tente `atleta.nivel = "expert"` (valor inválido) e confirme o `ValueError`. Depois adicione um novo nível válido `"profissional"` de forma correta (editando `NIVEIS_VALIDOS`, não contornando a validação).

**3.2** — Refatore uma parte do seu código do `Library_Dy` ou `Cleitinho_ChatBot` (ou, se preferir, descreva por escrito) identificando um atributo que hoje é público e não deveria ser — que invariante ele deveria respeitar, e como você encapsularia isso com `@property`.

# Miniprojeto — Parte 1: Sistema de Check-in de Academia

Sem gabarito pronto — resolva antes de seguir pra Parte 2.

## Especificação

Construa uma classe `Aluno` com:

- `nome` (público, sem restrição)
- `_creditos` (encapsulado): nunca pode ficar negativo. Método `fazer_checkin()` consome 1 crédito — se `_creditos == 0`, lança uma exceção customizada `SemCreditosError` (crie essa exceção herdando de `Exception`).
- `_peso` (encapsulado via `@property`): sempre positivo, e além disso **não pode variar mais de 5kg de uma vez** (regra de negócio simulando proteção contra erro de digitação — se alguém tentar `aluno.peso = aluno.peso + 20` de uma vez, deve dar erro).
- Uma `property` somente-leitura `pode_treinar` que retorna `True/False` baseado em `_creditos > 0`.

Teste seu código tentando ativamente quebrar as invariantes (crédito negativo, peso absurdo, salto de peso grande) e confirme que tudo é bloqueado corretamente antes de seguir em frente.

## **PARTE 2 — POLIMORFISMO**

# Capítulo 4 — Duck typing: polimorfismo sem hierarquia

## Conceito

“Se anda como pato e grasna como pato, é um pato” — em Python, o que importa pra um objeto poder ser usado em determinado contexto **não é o tipo dele, é se ele tem o comportamento (método/atributo) esperado**. Isso é diferente de linguagens com tipagem estática forte, onde polimorfismo quase sempre depende de herança/interfaces declaradas.

```
class Nadador:
    def treinar(self):
        return "nadando"

class Halterofilista:
    def treinar(self):
        return "levantando peso"

class Corda:
    """Nem é um 'atleta' conceitualmente, mas tem o método 'treinar' — e isso já basta."""
    def treinar(self):
        return "pulando corda"

def sessao(coisa_que_treina):
    print(coisa_que_treina.treinar())

for x in [Nadador(), Halterofilista(), Corda()]:
    sessao(x) # funciona pros três, sem herança comum nenhuma
```

Repara: Nadador, Halterofilista e Corda **não compartilham nenhuma superclasse**. sessao() não se importa com o tipo — só se importa que o objeto responde a .treinar(). Isso é polimorfismo por duck typing.

## Duck typing vs polimorfismo por herança — quando usar cada um

|          | Duck typing                           | Polimorfismo com herança comum                    |
|----------|---------------------------------------|---------------------------------------------------|
| Contrato | Implícito (só convenção/documentação) | Explícito (a superclasse ou ABC declara o método) |

|               | Duck typing                                                             | Polimorfismo com herança comum                                                   |
|---------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Flexibilidade | Alta — qualquer objeto com o método certo serve                         | Menor — precisa herdar da hierarquia                                             |
| Segurança     | Erro só aparece em runtime se o método não existir                      | Erro aparece na criação, se usar ABC (capítulo 8 do curso principal)             |
| Quando usar   | Scripts pequenos, prototipagem, quando a relação entre os tipos é fraca | Sistemas maiores onde o contrato precisa ser garantido e documentado formalmente |

Na prática, um sistema maduro costuma combinar os dois: usa ABC pra formalizar contratos importantes, mas ainda se beneficia de duck typing em pontos onde a flexibilidade vale mais que a garantia formal.

## hasattr e EAFP vs LBYL

Duas filosofias pra lidar com “esse objeto tem esse método mesmo?”:

```
# LBYL – Look Before You Leap (checar antes)
if hasattr(objeto, "treinar"):
    objeto.treinar()
else:
    print("objeto não sabe treinar")

# EAFP – Easier to Ask Forgiveness than Permission (tentar e tratar erro)
try:
    objeto.treinar()
except AttributeError:
    print("objeto não sabe treinar")
```

**Python prefere EAFP culturalmente** — é considerado mais “pythonico” tentar e capturar a exceção do que checar antes, entre outros motivos porque `hasattr` tem suas próprias armadilhas sutis (ex: pode mascarar outros erros dependendo da versão do Python) e porque, entre o `hasattr` checar e você efetivamente chamar o método, tecnicamente o estado do sistema já pode ter mudado em cenários concorrentes.

## Erros comuns

- Superusar duck typing em sistemas grandes sem nenhuma documentação do contrato esperado — funciona até alguém passar um objeto que não tem o método certo, e o erro só aparece em produção, na hora de rodar, não antes.
- Confundir duck typing com “não precisar pensar em design” — você ainda precisa garantir, de algum jeito (teste, documentação, type hints com Protocol), que os objetos que chegam ali de fato têm o comportamento esperado.

## Exercícios

**4.1** — Crie três classes sem nenhuma relação de herança entre si (Bicicleta, Esteira, Elastico), cada uma com um método `usar()` retornando uma string diferente. Escreva uma função que recebe uma lista misturando as três e chama `.usar()` em cada uma.

**4.2** — Usando `try/except AttributeError` (estilo EAFP), escreva uma função `treinar_com_seguranca(obj)` que tenta chamar `objeto.treinar()` e, se o objeto não tiver esse método, imprime uma mensagem amigável em vez de deixar o programa quebrar.

# Capítulo 5 — Sobrescrita de métodos (override) e polimorfismo com herança

## Conceito

Quando uma subclasse redefine um método que já existe na superclasse, com a mesma assinatura, isso é **sobrescrita (override)** — é a forma de polimorfismo que você já viu no capítulo 6/7 do curso principal, mas agora vamos ver os detalhes finos que geram bug quando ignorados.

```
class Atleta:
    def treinar(self, intensidade="normal"):
        return f"treino {intensidade} genérico"

class Nadador(Atleta):
    def treinar(self, intensidade="normal"):
        base = super().treinar(intensidade) # reaproveita o comportamento base
        return f"{base} — na piscina"

class Iniciante(Atleta):
    def treinar(self, intensidade="normal"):
        # Ignora completamente o parâmetro recebido — sempre força "leve"
        # Isso é uma DECISÃO válida, mas precisa ser deliberada, não acidental
        return super().treinar("leve")
```

## Cuidado: mudar a assinatura na sobrescrita quebra o polimorfismo

```
class Atleta:
    def treinar(self, intensidade="normal"):
        return f"treino {intensidade}"

class NadadorRuim(Atleta):
    def treinar(self, estilo, intensidade="normal"): # PARÂMETRO EXTRA OBRIGATÓRIO
        return f"nadando {estilo} — {intensidade}"

def sessao(atleta):
```

```
print(atleta.treinar()) # assume a assinatura da superclasse

sessao(A atleta()) # OK
sessao(NadadorRuim()) # TypeError! Faltou o argumento 'estilo' obrigatório
```

Isso é exatamente o que o princípio de **Liskov Substitution** (capítulo 11 do curso principal) proíbe: uma subclasse não deveria exigir mais, ou prometer menos, do que a superclasse promete — porque isso quebra qualquer código escrito pensando em “qualquer Atleta”, que é justamente o ponto de existir polimorfismo.

## isinstance() e subclass() — quando checar tipo é legítimo

Polimorfismo bem feito evita `if isinstance(...)` espalhado pelo código (isso é o “cheiro” de Open/Closed violado, capítulo 11 do curso principal). Mas existem usos legítimos:

```
# Legítimo: validação de entrada, não lógica de negócio ramificada
def adicionar_atleta(self, atleta):
    if not isinstance(atleta, Atleta):
        raise TypeError("Esperado um Atleta")
    self.membros.append(atleta)
```

```
# ILEGÍTIMO — isso é o que polimorfismo deveria substituir
```

```
def treinar_qualquer(atleta):
    if isinstance(atleta, Nadador):
        return "nadando"
    elif isinstance(atleta, Halterofilista):
        return "levantando"
```

```
# toda vez que criar um tipo novo, tem que voltar aqui e adicionar um elif — viola Open
```

## Erros comuns

- Esquecer `super().metodo()` quando o comportamento da superclasse ainda deveria acontecer, e sobrescrever “substituindo” quando a intenção real era “estender”.
- Mudar a assinatura do método sobrescrito (parâmetros extras obrigatórios, tipo de retorno incompatível) — quebra silenciosamente qualquer código polimórfico.
- Usar `isinstance()` em cadeia como substituto de polimorfismo de verdade.

## Exercícios

**5.1** — Refaça o `NadadorRuim` do exemplo para `NadadorCorreto`, mantendo intensidade como parâmetro compatível e adicionando `estilo` como parâmetro **opcional** (com valor padrão), de forma que `sessao(NadadorCorreto())` continue funcionando sem erro.

**5.2** — Encontre (ou escreva de propósito) um trecho de código com `if isinstance(x, TipoA): ... elif isinstance(x, TipoB): ...` fazendo lógica de negócio ramificada, e refatore usando polimorfismo (sobrescrita de método), eliminando os `isinstance`.

# Capítulo 6 — Sobrecarga de operadores (operator overloading)

## Conceito

Sobrecarga de operadores é uma forma específica de polimorfismo: você define o que `+`, `==`, `<`, `len()`, etc. significam para os **seus próprios tipos**, através dos métodos mágicos (dunder) vistos no capítulo 9 do curso principal.

```
class CargaTreino:
    def __init__(self, peso_kg, series, repeticoes):
        self.peso_kg = peso_kg
        self.series = series
        self.repeticoes = repeticoes

    @property
    def volume(self):
        return self.peso_kg * self.series * self.repeticoes

    def __add__(self, outra):
        """Define o que CargaTreino + CargaTreino significa."""
        if not isinstance(outra, CargaTreino):
            return NotImplemented # deixa o Python tentar outras formas, ou dar TypeError
        return CargaTreino(
            peso_kg=(self.peso_kg + outra.peso_kg) / 2, # média, é uma decisão de design
            series=self.series + outra.series,
            repeticoes=self.repeticoes + outra.repeticoes,
        )

    def __gt__(self, outra):
        return self.volume > outra.volume

    def __repr__(self):
        return f"CargaTreino({self.peso_kg}kg, {self.series}x{self.repeticoes})"

treino_a = CargaTreino(80, 4, 8)
treino_b = CargaTreino(60, 3, 10)

combinado = treino_a + treino_b # chama __add__
print(combinado)               # CargaTreino(70.0kg, 7x18)
```

```
print(treino_a > treino_b)          # chama __gt__, compara por volume
```

## NotImplemented vs NotImplementedError — não confunda

Isso é um erro sutil e comum:

- `NotImplemented` (sem parênteses, é um valor/singleton) — retornado dentro de dunder methods quando a operação não faz sentido para o tipo recebido, deixando o Python tentar o caminho reverso (`__radd__`) ou finalmente lançar `TypeError` de forma clara.
- `NotImplementedError` (uma exceção, com parênteses) — levantada (`raise`) tipicamente em métodos abstratos que uma subclasse deveria ter implementado (embora `@abstractmethod` do módulo `abc` seja a forma mais robusta de garantir isso, como vimos no capítulo 8 do curso principal).

```
def __add__(self, outra):
    if not isinstance(outra, CargaTreino):
        return NotImplemented  # correto aqui — não é erro nosso, é operação não suportada
```

## Tabela de dunder methods de operadores mais usados

| Operador            | Método                                                                                               |
|---------------------|------------------------------------------------------------------------------------------------------|
| +                   | <code>__add__</code> (e <code>__radd__</code> para o caso reverso)                                   |
| -                   | <code>__sub__</code>                                                                                 |
| *                   | <code>__mul__</code>                                                                                 |
| ==                  | <code>__eq__</code>                                                                                  |
| !=                  | <code>__ne__</code> (Python 3 já deriva automaticamente de <code>__eq__</code> na maioria dos casos) |
| <, <=, >, >=        | <code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>                |
| <code>len(x)</code> | <code>__len__</code>                                                                                 |
| <code>in</code>     | <code>__contains__</code>                                                                            |
| <code>x()</code>    | <code>__call__</code>                                                                                |

**Dica prática:** você raramente precisa implementar todos os operadores de comparação manualmente. O decorator `@total_ordering` do módulo `functools` gera os outros automaticamente a partir de `__eq__` + um entre `__lt__`/`__le__`/`__gt__`/`__ge__`.

```
from functools import total_ordering
```

```
@total_ordering
class CargaTreino:
    def __eq__(self, outra):
        return self.volume == outra.volume

    def __lt__(self, outra):
        return self.volume < outra.volume

# __le__, __gt__, __ge__ são gerados automaticamente pelo decorator
```

## Erros comuns

- Implementar `__eq__` sem cuidar de `__hash__` (já visto no capítulo 9 do curso principal) — mencionado de novo aqui porque em sobrecarga de operadores é onde o problema mais aparece na prática.
- Retornar `NotImplementedError` (a exceção) em vez de `NotImplemented` (o valor) dentro de um dunder de operador — isso quebra o mecanismo de fallback do Python e gera uma mensagem de erro pior pro usuário final da sua classe.
- Sobrecarregar operador com semântica que confunde quem lê (ex: `+` fazendo subtração “por engraçadinho”) — sobrecarga de operador deve ser **intuitiva**, senão é pior que não ter.

## Exercícios

**6.1** — Implemente `__add__` em `PlanoDeTreino` de forma que `plano_a + plano_b` retorne um novo `PlanoDeTreino` contendo os exercícios de ambos combinados.

**6.2** — Use `@total_ordering` numa classe `CargaTreino` própria sua, implementando só `__eq__` e `__lt__`, e confirme que `>`, `<=` e `>=` funcionam mesmo sem você tê-los escrito.

**6.3 (desafio)** — Implemente `__call__` em uma classe `ContadorDeRepeticoes`, de forma que `contador()` incremente e retorne a contagem atual — permitindo usar o objeto como se fosse uma função (`contador()`, `contador()`, `contador()` incrementando a cada chamada).

# Capítulo 7 — Polimorfismo com ABC: contrato formal + comportamento variável

## Conceito

Combinando o capítulo 8 do curso principal (ABC) com polimorfismo: uma classe abstrata pode declarar um método abstrato que **força** cada subclasse a fornecer sua própria implementação, e o código que consome essas classes nunca precisa saber qual subclasse específica está em mãos — o melhor dos dois mundos: contrato garantido + flexibilidade.

```
from abc import ABC, abstractmethod
```

```
class Exercicio(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def calcular_volume(self):
        ...

    def descricao_completa(self):
        """Método concreto, comum a todos — não precisa ser reescrito, mas PODE ser."""
        return f"{self.nome}: volume = {self.calcular_volume()}"

class ExercicioForca(Exercicio):
    def __init__(self, nome, peso_kg, series, repeticoes):
        super().__init__(nome)
        self.peso_kg = peso_kg
        self.series = series
        self.repeticoes = repeticoes

    def calcular_volume(self):
        return self.peso_kg * self.series * self.repeticoes

class ExercicioCardio(Exercicio):
    def __init__(self, nome, distancia_km, ritmo_min_km):
        super().__init__(nome)
        self.distancia_km = distancia_km
        self.ritmo_min_km = ritmo_min_km
```

```

def calcular_volume(self):
    return self.distancia_km # decisão de design: volume de cardio = distância

exercicios = [
    ExercicioForca("Supino", 80, 4, 8),
    ExercicioCardio("Corrida", 5, 5.5),
]

for ex in exercicios:
    print(ex.descricao_completa()) # polimorfismo: cada um calcula volume do seu jeito

```

Isso é o padrão mais comum em código profissional: ABC define o contrato mínimo, subclasses concretas implementam a variação, e código cliente (o for acima) nunca precisa de `if isinstance`.

## Erros comuns

- Esquecer que métodos concretos de uma ABC (como `descricao_completa` acima) **podem** ser sobrescritos por subclasses se fizer sentido — ABC não trava tudo, só o que é explicitamente `@abstractmethod`.

## Exercícios

**7.1** — Adicione uma terceira subclasse `ExercicioFlexibilidade(Exercicio)` (alongamento, por exemplo) com sua própria lógica de `calcular_volume` (pode ser `tempo_segundos / 60`, por exemplo). Confirme que o `for` do exemplo acima funciona sem nenhuma modificação — esse é o teste real de que seu polimorfismo está bem desenhado.

# Projeto integrador final — Encapsulamento + Polimorfismo juntos

Esse é o projeto que junta tudo dos dois documentos. Não tem prazo, mas não pula — é aqui que a teoria vira competência de verdade.

## Especificação: Sistema de Ranking de Academia

1. Exercício como ABC, com `calcular_volume()` abstrato — pelo menos 3 subclasses concretas diferentes (força, cardio, e uma terceira à sua escolha).
2. Atleta com:
  - nome público.
  - `_peso` encapsulado via `@property`, com validação (positivo, e variação máxima de 5kg por atribuição, igual ao miniprojeto da Parte 1).
  - `_creditos` encapsulado, nunca negativo, com `SemCreditosError` customizada se tentar treinar sem crédito.
  - Uma lista (composição) de Exercicios realizados.
  - Método `fazer_treino(self, exercicio)` que **só executa se `_creditos > 0`**, consome 1 crédito, e adiciona o exercício à lista.
  - Property somente-leitura `volume_total` somando `calcular_volume()` de todos os exercícios feitos — repare que isso é polimorfismo puro: Atleta não sabe (nem precisa saber) que tipo de exercício está somando.
3. Sobrecarga de operadores em Atleta: `__gt__`/`__eq__` comparando por `volume_total` (use `@total_ordering` se quiser economizar código), e `__str__` formatado.
4. Uma função `ranking(lista_atletas)` usando `sorted()`, sem nenhum `isinstance` — deve funcionar independente de quais tipos de Exercicio cada atleta treinou.
5. Bônus: implemente `__len__` em Atleta retornando a quantidade de exercícios feitos, permitindo `len(atleta)`.

## Checklist de autoavaliação

- ☐ É impossível, através da API pública, colocar Atleta num estado inválido (peso negativo, crédito negativo, salto de peso absurdo)?
- ☐ Existe algum `if isinstance(exercicio, ...)` no seu código de Atleta ou ranking? Se sim, isso é sinal de que o polimorfismo não está bem desenhado — deveria ser zero.
- ☐ Se você adicionar uma quarta subclasse de Exercicio amanhã, precisa mexer em Atleta ou em ranking? A resposta certa é não.

- ☐ Você consegue explicar, em voz alta, sem olhar o código, a diferença entre onde usou encapsulamento e onde usou polimorfismo nesse projeto? Se travar, volte no capítulo correspondente antes de seguir.

# Apêndice — Gabarito comentado (só depois de tentar)

## Capítulo 1

### 1.1

```
class Cofre:
    def __init__(self, senha):
        self.__senha = senha

cofre = Cofre("1234")
# cofre.__senha -> AttributeError: 'Cofre' object has no attribute '__senha'
print(cofre._Cofre__senha) # '1234' - funciona
```

Prova que não é privado “de verdade”: o name mangling só troca o nome (\_Cofre\_\_senha), não bloqueia o acesso — é ofuscação, não segurança.

### 1.2

```
class ContaTreino:
    def __init__(self, saldo_creditos):
        self._saldo_creditos = saldo_creditos

    def adicionar_creditos(self, qtd):
        if qtd <= 0:
            raise ValueError("Quantidade deve ser positiva")
        self._saldo_creditos += qtd

    def usar_credito(self):
        if self._saldo_creditos <= 0:
            raise ValueError("Sem créditos disponíveis")
        self._saldo_creditos -= 1
```

## Capítulo 2

### 2.1

```
class Atleta:
    def __init__(self, nome, peso, altura):
        self.nome = nome
        self.peso = peso # já valida via setter
        self.altura = altura # já valida via setter
```

```

@property
def peso(self):
    return self._peso

@peso.setter
def peso(self, valor):
    if valor <= 0:
        raise ValueError("Peso precisa ser positivo")
    self._peso = valor

@property
def altura(self):
    return self._altura

@altura.setter
def altura(self, valor):
    if valor <= 0:
        raise ValueError("Altura precisa ser positiva")
    self._altura = valor

```

**2.3** — O código com bug:

```

class AtletaBuggy:
    @property
    def peso(self):
        return self.peso          # chama a si mesma -> recursão

    @peso.setter
    def peso(self, valor):
        self.peso = valor        # idem

```

Corrigido: sempre guardar em `self._peso` (nome diferente) dentro do getter/setter, nunca `self.peso` dentro da própria property peso.

## Capítulo 4

### 4.2

```

def treinar_com_seguranca(objeto):
    try:
        print(objeto.treinar())
    except AttributeError:
        print(f"{objeto} não sabe treinar")

```

## Capítulo 6

### 6.2

```

from functools import total_ordering

@total_ordering
class CargaTreino:

```

```

def __init__(self, peso_kg, series, repeticoes):
    self.peso_kg = peso_kg
    self.series = series
    self.repeticoes = repeticoes

@property
def volume(self):
    return self.peso_kg * self.series * self.repeticoes

def __eq__(self, outra):
    return self.volume == outra.volume

def __lt__(self, outra):
    return self.volume < outra.volume

a = CargaTreino(80, 4, 8)
b = CargaTreino(60, 3, 10)
print(a > b, a <= b, a >= b) # todos funcionam, sem implementar __gt__/__le__/__ge__

```

### 6.3

```

class ContadorDeRepeticoes:
    def __init__(self):
        self._contagem = 0

    def __call__(self):
        self._contagem += 1
        return self._contagem

contador = ContadorDeRepeticoes()
print(contador()) # 1
print(contador()) # 2
print(contador()) # 3

```